Guía de la implementación en Godot

Para los expositores · Colisiones en videojuegos · Estructura de Datos II (IST 4031) · Universidad del Norte · 2026-30
María Yusti · Axel Tornay · Juan Vizcaíno · Isac Monterrosa

Para qué sirve este documento. Explica la carpeta `ejemplo-colision-videojuegos` sin dejar nada por fuera: qué es, para qué se hizo, cómo se construyó, qué hace cada archivo, nodo, variable y función, cómo ejecutarlo, cómo demostrarlo y qué preguntas pueden hacerles. Todo lo que aquí se dice sobre nombres, valores y código es **copia fiel de los archivos reales**; los números de línea corresponden a los archivos tal como están en la carpeta.
Estado de verificación en una línea: las 17 pruebas manuales se ejecutaron en Godot 4.7.2 (headless) sin fallos y el proyecto cargó sin errores; la escena se grabó corriendo (pelotas, rebotes, contadores y AABB funcionan); **no se ha visto aún** el movimiento del jugador con teclado ni la recogida de monedas.

Índice

- 1. Qué es la implementación y qué no es
- 2. Objetivo
- 3. Qué ve y hace quien la ejecuta
- 4. Cómo se construyó, paso a paso
- 5. Estructura de archivos
- 6. Configuración del proyecto (`project.godot`)
- 7. Las escenas, nodo por nodo
- 8. Los scripts, función por función
- 9. Qué pasa en cada cuadro (flujo completo y mapa de señales)
- 10. Las 17 pruebas manuales, caso por caso
- 11. Cómo ejecutarlo y qué comandos usar
- 12. Guion de la demostración en vivo
- 13. Qué está verificado y qué no
- 14. Límites de la implementación
- 15. Preguntas probables sobre el código
- 16. Glosario
- 17. Autoría y fuentes

1. Qué es la implementación y qué no es

Qué es. Un **laboratorio de colisiones en 2D** hecho en Godot 4.7 (lenguaje GDScript, renderizador Forward Plus). Es una sola escena jugable donde conviven los cuatro tipos de objeto de colisión que ofrece Godot en 2D, más unas funciones de prueba de colisión escritas a mano, para **mostrar en funcionamiento** lo que se explica en la presentación: detección frente a respuesta, formas de colisión, fase estrecha, cajas envolventes (AABB), capas y máscaras, y grafos de contacto.

De dónde parte. La carpeta original era una plantilla vacía de Godot: solo tenía `project.godot` e `icon.svg`. Se reutilizó esa configuración (versión 4.7, Forward Plus, ventana, etc.) y se agregaron las escenas y scripts. No se borró nada original.

Qué NO es (conviene decirlo antes de que lo pregunten):

- No es un videojuego completo: no hay niveles, puntaje final ni fin de partida.

- No reemplaza ni reimplementa el motor de física de Godot: la detección y el rebote reales los hace Godot. Las funciones de `colision_manual.gd` son una **versión didáctica** de las pruebas de la fase estrecha, usadas para comparar y para las pruebas numéricas.
- No implementa una fase amplia propia (QuadTree, cuadrícula o sweep and prune). Esas estructuras se explican como teoría; Godot ya trae la suya.
- No usa detección continua (CCD): las pelotas usan el modo por defecto.
- No usa hilos ni multijugador.

2. Objetivo

Objetivo general. Demostrar, con una aplicación ejecutable, cómo se detectan y se responden las colisiones en un videojuego, y relacionar esa práctica con la teoría de la presentación y con los temas del curso.

Objetivo específico	Dónde se cumple	Diapositiva
Distinguir **detectar** de **responder**	Moneda (solo detecta), Pelota (responde el motor), Jugador (responde el código)	5, 15
Mostrar distintas **formas de colisión**	Rectángulo (jugador y paredes), círculo (pelota y moneda), polígono convexo (obstáculo triangular)	6
Mostrar la **fase estrecha** con trazas concretas	`colision_manual.gd` y sus 17 pruebas	7, 8
Mostrar las **cajas envolventes (AABB)** como filtro	Rectángulos cian dibujados sobre jugador y pelotas (tecla Espacio)	6, 11
Mostrar **capas y máscaras**	Cuatro capas con nombre y los valores de `collision_layer` / `collision_mask`	16
Relacionar con **grafos** (tema de clase)	Grupos de cuerpos en contacto con DFS en `laboratorio.gd`	12
Comparar la prueba manual con el motor	Última línea del panel: «prueba manual: sí/no | motor: sí/no»	18
Cumplir la rúbrica (I1, I2, I3)	Código explicable, ejecución en vivo con casos, fuentes declaradas	17, 18

3. Qué ve y hace quien la ejecuta

Al pulsar F5 se abre una ventana de 1280×720 con un fondo gris oscuro y un borde más oscuro (las cuatro paredes). Se ven:

- Un **cuadrado azul** (el jugador), a la izquierda. Se pone **rojo** mientras está tocando algo.
- Tres **círculos naranja** (las pelotas) que se mueven solos y rebotan. Parpadean en **blanco** 0,15 s cada vez que chocan.
- Tres **círculos amarillos** (las monedas), quietas.
- Un **triángulo gris azulado** con contorno blanco en el centro (el obstáculo, rotado 15°).
- Un **panel de texto** arriba a la izquierda con los contadores.
- Rectángulos **cian** alrededor del jugador y de cada pelota (las AABB), y líneas **rosa** entre cuerpos que están en contacto.

Controles: flechas o WASD mueven al jugador · **Espacio** (o Enter) muestra u oculta las AABB y las líneas de contacto · **Esc** reinicia la escena.

El panel de texto muestra, línea por línea:

Línea del panel	Qué cuenta	Quién actualiza el valor
Choques del jugador (CharacterBody2D): n	Veces que el jugador *empieza* a tocar otro cuerpo	Señal `choque` de `Jugador`
Rebotes de pelotas (RigidBody2D): n	Veces que una pelota empieza a tocar otro cuerpo	Señal `rebote` de cada `Pelota`
Monedas recogidas (Area2D): n de 3	Monedas que el jugador tocó	Señal `recogida` de cada `Moneda`
Grupos de cuerpos en contacto (DFS): n	Componentes conexas del grafo jugador + pelotas	`_contar_grupos()` cada cuadro de física
Pelota 1 vs jugador -> prueba manual: sí/no | motor: sí/no	Si la prueba manual círculo–AABB y el motor dicen que Pelota 1 toca al jugador	`_physics_process` de `Laboratorio`

Detalle que pueden preguntar: cuando dos pelotas chocan entre sí, **cada una** emite su señal `body_entered`, así que el contador de rebotes sube de a 2 en ese evento. Es una consecuencia del diseño (cada pelota cuenta su propio contacto), no un error del motor. Además el contador de choques del jugador subió en la grabación sin que se pulsara ninguna tecla (valores 1, 2, 4), porque el jugador también registra los golpes de las pelotas contra él.

4. Cómo se construyó, paso a paso

Este es el orden en que se armó el proyecto, con el motivo de cada decisión. (El código se elaboró con asistencia de IA a partir de las instrucciones del grupo; ver sección 17.)

Paso	Qué se hizo	Por qué
1	Se inspeccionó la carpeta existente: solo `project.godot` e `icon.svg` (plantilla vacía de Godot 4.7).	Reutilizar la versión y la configuración sin romper nada.
2	Se editó `project.godot`: escena principal, tamaño de ventana 1280×720 y nombres de las capas de física.	Que F5 abra directamente el laboratorio y que las capas sean legibles.
3	Se crearon las **escenas pequeñas**: paredes, obstáculo, jugador, pelota, moneda. Cada una con su tipo de cuerpo y su forma.	Una escena por tipo de objeto permite reutilizarla (las 3 pelotas y las 3 monedas son instancias).
4	Se creó la escena principal `laboratorio.tscn`, que instancia todo, fija posiciones y velocidades iniciales y agrega la interfaz (`CanvasLayer` + `Label`).	Un solo punto de entrada; las posiciones se ven en un único archivo.
5	Se escribieron los scripts de cada objeto, con **señales** para avisar eventos (choque, rebote, recogida).	Desacoplar: el objeto avisa y el laboratorio solo cuenta; ningún objeto conoce al panel.
6	Se escribió `colision_manual.gd` con las cuatro pruebas de la fase estrecha.	Tener el algoritmo de la teoría a la vista y poder compararlo con el motor.
7	Se escribió `laboratorio.gd`: contadores, AABB, grafo de contactos con DFS y comparación manual/motor.	Conectar la práctica con los temas de clase (grafos) y con la teoría (AABB, fase estrecha).
8	Se escribió `pruebas_manuales.gd` con 17 comprobaciones calculadas a mano.	Verificar la matemática sin abrir el editor y tener una salida «Pruebas: 17 | fallos: 0».
9	Verificación: sintaxis con gdtoolkit; réplica en Python de las pruebas; ejecución en Godot 4.7.2 (pruebas y carga); grabación de cuadros de la escena.	Ver sección 13.

5. Estructura de archivos

```
ejemplo-colision-videojuegos/
├─ project.godot
├─ icon.svg                  (original de la plantilla)
├─ README_Godot.md
├─ escenas/
│   ├─ laboratorio.tscn      escena principal (Node2D)
│   ├─ paredes.tscn          StaticBody2D
│   ├─ obstaculo.tscn        StaticBody2D
│   ├─ jugador.tscn          CharacterBody2D
│   ├─ pelota.tscn           RigidBody2D
│   └─ moneda.tscn           Area2D
└─ scripts/
    ├─ laboratorio.gd        144 líneas
    ├─ jugador.gd             62
    ├─ pelota.gd              47
    ├─ moneda.gd              30
    ├─ paredes.gd             13
    ├─ obstaculo.gd           15
    ├─ superposicion.gd       14
    ├─ colision_manual.gd    125
    └─ pruebas_manuales.gd    95
```

Las escenas (`.tscn`) son archivos de texto que describen nodos y sus propiedades; los scripts (`.gd`) son GDScript. Los scripts usan **tabulaciones** para indentar (convención de Godot); en este documento se muestran con espacios.

6. Configuración del proyecto (`project.godot`)

Clave	Valor	Qué hace / por qué
`config/name`	«Ejemplo colision videojuegos»	Nombre del proyecto (original).
`run/main_scene`	`res://escenas/laboratorio.tscn`	**Agregado.** Escena que se abre con F5.
`config/features`	«4.7», «Forward Plus»	Versión del motor y renderizador (original, se conservó).
`window/size/viewport_width` y `_height`	1280 y 720	**Agregado.** Tamaño de la ventana; coincide con el tamaño de las paredes.
`window/stretch/mode` y `aspect`	«canvas_items» y «expand»	Escalado de la ventana (original).
`[layer_names]` 2d_physics/layer_1…4	«paredes», «jugador», «pelotas», «monedas»	**Agregado.** Nombres de las capas de física 2D, para que en el editor se lean con nombre.
`3d/physics_engine`	«Jolt Physics»	Original; no afecta a este proyecto 2D.
`rendering_device/driver.windows`	«d3d12»	Original.

7. Las escenas, nodo por nodo

7.1 `laboratorio.tscn` — escena principal

Árbol de nodos:

```
Laboratorio (Node2D, script laboratorio.gd)
├─ Paredes        (instancia de paredes.tscn)
├─ Obstaculo      (instancia, posición (640, 380))
├─ Jugador        (instancia, posición (160, 360))
├─ Pelotas        (Node2D contenedor)
│   ├─ Pelota1    (instancia, pos (900,180), velocidad_inicial (280,190))
│   ├─ Pelota2    (instancia, pos (400,560), velocidad_inicial (-230,-210))
│   └─ Pelota3    (instancia, pos (1000,560), velocidad_inicial (-300,-120))
├─ Monedas        (Node2D contenedor)
│   ├─ Moneda1    (instancia, pos (420,180))
│   ├─ Moneda2    (instancia, pos (860,380))
│   └─ Moneda3    (instancia, pos (540,600))
├─ Superposicion  (Node2D, z_index = 10, script superposicion.gd)
└─ Interfaz       (CanvasLayer)
    └─ Texto      (Label)
```

- **Contenedores `Pelotas` y `Monedas`** (Node2D vacíos): agrupan instancias; `Monedas` se recorre con `$Monedas.get_children()`.
- **`velocidad_inicial`** de cada pelota se fija *desde la escena* (variable exportada), así cada pelota arranca en una dirección distinta sin tocar el script.
- **`Superposicion`** con `z_index = 10`: se dibuja por encima del resto del mundo; ahí se pintan las AABB y las líneas de contacto.
- **`Interfaz` (CanvasLayer) y `Texto` (Label)**: el texto va en una capa de interfaz aparte del mundo. El Label mide 744×190 px desde (36, 30), con fuente blanca de tamaño 17.

7.2 `paredes.tscn`

Nodo / propiedad	Valor	Explicación
Paredes — StaticBody2D	script `paredes.gd`	Cuerpo estático: participa en las colisiones pero el motor nunca lo mueve.
`physics_material_override`	PhysicsMaterial: friction 0, bounce 1	Las pelotas rebotan sin perder energía y sin roce.
Arriba — CollisionShape2D	pos (640, 10), RectangleShape2D 1280×20	Pared superior.
Abajo	pos (640, 710), 1280×20	Pared inferior (20 px visibles: 700 a 720).
Izquierda	pos (10, 360), RectangleShape2D 20×720	Pared izquierda.
Derecha	pos (1270, 360), 20×720	Pared derecha.
Capa / máscara	No se cambiaron: capa 1 «paredes», máscara 1 (valores por defecto)	Por eso están en la capa «paredes».

Un solo cuerpo estático con cuatro formas hijas es la forma habitual de armar un marco; cada `CollisionShape2D` aporta una forma al mismo cuerpo.

7.3 `obstaculo.tscn`

Nodo / propiedad	Valor	Explicación
Obstaculo — StaticBody2D	rotation 0,261799 rad (15°); script `obstaculo.gd`	Cuerpo estático rotado: su AABB sería mayor que la forma real.
`physics_material_override`	friction 0, bounce 1	Rebote perfecto.
Forma — CollisionPolygon2D	polygon (0,−60), (70,50), (−70,50)	Triángulo **convexo**. Es el ejemplo de «polígono convexo» de la teoría.

7.4 `jugador.tscn`

Nodo / propiedad	Valor	Explicación
Jugador — CharacterBody2D	grupo «jugador»; script `jugador.gd`	Cuerpo que se mueve por código. El grupo permite que la moneda lo reconozca.
`collision_layer`	2	Está en la capa 2, «jugador».
`collision_mask`	5 (bits 1 y 3)	Choca con la capa 1 (paredes y obstáculo) y la capa 3 (pelotas). No escanea monedas.
Forma — CollisionShape2D	RectangleShape2D 40×40	Cuadrado de 40 px.

7.5 `pelota.tscn`

Nodo / propiedad	Valor	Explicación
Pelota — RigidBody2D	grupo «pelotas»; script `pelota.gd`	Cuerpo con física: el motor calcula la respuesta.
`collision_layer`	4 (bit 3)	Está en la capa 3, «pelotas».
`collision_mask`	7 (bits 1, 2 y 3)	Choca con paredes/obstáculo, jugador y otras pelotas.
`physics_material_override`	friction 0, bounce 1	Rebote perfecto sin roce.
`gravity_scale`	0.0	Sin gravedad: es una vista cenital, las pelotas no caen.
`can_sleep`	false	Impide que el motor «duerma» el cuerpo cuando casi no se mueve; así nunca se detiene.
`contact_monitor`	true	**Necesario** para que emita `body_entered` y para usar `get_colliding_bodies()`.
`max_contacts_reported`	8	Cantidad máxima de contactos que guarda. Sin un valor mayor que 0 no se reportan contactos.
`linear_damp_mode` y `angular_damp_mode`	1 (Replace)	El amortiguamiento del cuerpo (que queda en su valor por defecto, 0) sustituye al del proyecto, para que la pelota no vaya perdiendo velocidad.
Forma — CollisionShape2D	CircleShape2D radio 20	Círculo de 20 px de radio.

7.6 `moneda.tscn`

Nodo / propiedad	Valor	Explicación
Moneda — Area2D	script `moneda.gd`	Zona de detección: no bloquea nada, solo avisa.
`collision_layer`	8 (bit 4)	Está en la capa 4, «monedas».
`collision_mask`	2 (bit 2)	Solo escanea la capa 2, «jugador». Las pelotas la atraviesan sin activarla.
Forma — CollisionShape2D	CircleShape2D radio 16	Círculo de 16 px.

7.7 Resumen de capas y máscaras

Objeto	Está en la capa…	Escanea las capas…	Valores guardados (layer / mask)
Paredes y Obstáculo	1 «paredes»	1 (por defecto; irrelevante en un cuerpo estático)	no se guardan (por defecto 1 / 1)
Jugador	2 «jugador»	1 y 3 (paredes y pelotas)	2 / 5
Pelota	3 «pelotas»	1, 2 y 3 (paredes, jugador y pelotas)	4 / 7
Moneda	4 «monedas»	2 (jugador)	8 / 2

Los valores guardados son **máscaras de bits**: capa 1 = 1, capa 2 = 2, capa 3 = 4, capa 4 = 8. La máscara del jugador es 1 + 4 = 5; la de la pelota es 1 + 2 + 4 = 7.

8. Los scripts, función por función

8.1 `jugador.gd` — el CharacterBody2D

Idea: un `CharacterBody2D` detecta colisiones pero **no tiene física propia**: todo el movimiento y la respuesta los decide el código. Aquí la respuesta es **deslizar** contra lo que choque y **avisar** con una señal.

SCRIPTS/JUGADOR.GD · LÍNEAS 1–62

```
class_name Jugador
extends CharacterBody2D
## Jugador controlado con el teclado (CharacterBody2D).
##
## Un CharacterBody2D detecta colisiones, pero NO tiene física propia:
## todo el movimiento y la respuesta se programan. Aquí la respuesta es
## deslizarse (move_and_slide) y avisar con una señal cuando empieza un choque.

## Se emite una sola vez cuando el jugador empieza a tocar otro cuerpo.
signal choque(con: Node, normal: Vector2)

@export var velocidad: float = 320.0

var lado: float = 40.0

var _tocando: bool = false
var _tocaba_antes: bool = false

@onready var _forma: CollisionShape2D = $Forma


func _ready() -> void:
    lado = (_forma.shape as RectangleShape2D).size.x


func _physics_process(_delta: float) -> void:
    var direccion: Vector2 = Vector2(
        Input.get_axis("ui_left", "ui_right"),
        Input.get_axis("ui_up", "ui_down")
    )
    var d_tecla: float = float(Input.is_physical_key_pressed(KEY_D))
    var a_tecla: float = float(Input.is_physical_key_pressed(KEY_A))
    var s_tecla: float = float(Input.is_physical_key_pressed(KEY_S))
    var w_tecla: float = float(Input.is_physical_key_pressed(KEY_W))
    direccion += Vector2(d_tecla - a_tecla, s_tecla - w_tecla)
    direccion = direccion.limit_length(1.0)

    velocity = direccion * velocidad
    move_and_slide()

    # Respuesta a la colisión: revisar con qué se chocó en este paso.
    _tocando = get_slide_collision_count() > 0
    if _tocando and not _tocaba_antes:
        var info: KinematicCollision2D = get_slide_collision(0)
        choque.emit(info.get_collider(), info.get_normal())
    _tocaba_antes = _tocando
    queue_redraw()


## Caja envolvente (AABB) del jugador en coordenadas globales.
func caja_global() -> Rect2:
    var mitad: Vector2 = Vector2(lado, lado) / 2.0
    return Rect2(global_position - mitad, Vector2(lado, lado))


func _draw() -> void:
    var color: Color = Color("4da3ff")
    if _tocando:
        color = Color("ff5a5f")
    var mitad: Vector2 = Vector2(lado, lado) / 2.0
    draw_rect(Rect2(-mitad, Vector2(lado, lado)), color)
    draw_rect(Rect2(-mitad, Vector2(lado, lado)), Color.WHITE, false, 2.0)
```

Líneas	Qué hace
1–2	`class_name Jugador` le da un nombre global a la clase (así `laboratorio.gd` puede escribir `Jugador` como tipo). `extends CharacterBody2D`: el script *es* un CharacterBody2D.
3–7	Comentario de documentación (`##`): resume la idea.
9–10	`signal choque(con, normal)`: señal que se emite **una sola vez** cuando empieza un contacto. Lleva el cuerpo con el que chocó y la normal del contacto.
12	`@export var velocidad = 320.0`: velocidad en píxeles por segundo; `@export` la hace editable desde el inspector.
14	`lado = 40.0`: lado del cuadrado; se corrige en `_ready` leyendo la forma real.
16–17	`_tocando` (¿hay contacto en este cuadro?) y `_tocaba_antes` (¿había en el anterior?). Comparar ambos detecta el *inicio* del contacto.
19	`@onready var _forma = $Forma`: referencia al nodo hijo «Forma», asignada cuando el nodo está listo.
22–23	`_ready()`: toma el ancho del `RectangleShape2D` como `lado`. Así el dibujo y la AABB siempre coinciden con la forma de colisión.
26	`_physics_process(_delta)`: Godot lo llama en cada paso de física (60 por segundo por defecto).
27–29	Dirección base con las acciones predefinidas: `Input.get_axis("ui_left", "ui_right")` devuelve −1, 0 o 1 (flechas). Igual para vertical (`ui_up` , `ui_down`).
31–35	Suma las teclas WASD con `Input.is_physical_key_pressed(KEY_D)` etc. (cada una devuelve 0 o 1 al convertir a `float`) y las añade: derecha − izquierda en x, abajo − arriba en y.
36	`direccion.limit_length(1.0)`: evita que la diagonal (o pulsar flecha y WASD a la vez) sea más rápida.
38	`velocity = direccion * velocidad`: `velocity` es la propiedad de CharacterBody2D que `move_and_slide` usa.
39	`move_and_slide()`: **mueve el cuerpo** con esa velocidad y, si choca, lo **desliza** a lo largo de la superficie en vez de detenerlo. Es la «respuesta programada» del jugador.
41–42	`get_slide_collision_count() > 0`: ¿hubo contactos en este paso?
43–45	Si **hay contacto ahora y no lo había antes**: `get_slide_collision(0)` devuelve el primer contacto como `KinematicCollision2D`; de ahí se leen `get_collider()` (con quién) y `get_normal()` (normal del contacto) y se emite `choque`.
46–47	Guarda el estado para el próximo cuadro y pide redibujar (`queue_redraw()`) para que el color cambie.
51–53	`caja_global()`: devuelve la AABB del jugador como `Rect2` en coordenadas globales (centro ± mitad del lado). La usa `laboratorio.gd` para dibujar la AABB y para la prueba manual.
56–62	`_draw()`: dibuja el cuadrado azul (rojo si `_tocando`) y un contorno blanco de 2 px.

Frase para exponer: «El jugador es un CharacterBody2D, entonces Godot detecta el choque pero no decide qué hacer: nosotros movemos con `move_and_slide`, leemos la normal del contacto y emitimos una señal.»

8.2 `pelota.gd` — el RigidBody2D

Idea: un `RigidBody2D` **sí tiene física**. El código no calcula el rebote: solo le da una velocidad inicial y escucha el evento de contacto para contar y parpadear. El rebote lo calcula el motor con el `PhysicsMaterial` (bounce 1, friction 0).

SCRIPTS/PELOTA.GD · LÍNEAS 1–47

```gdscript
class_name Pelota
extends RigidBody2D
## Pelota con física (RigidBody2D).
##
## Un RigidBody2D SÍ tiene física: no se mueve con código, el motor calcula
## la respuesta (rebote). Aquí solo se le da una velocidad inicial y se
## escucha la señal body_entered para contar los rebotes.
## El rebote lo define el PhysicsMaterial (bounce = 1.0, friction = 0.0).

## Se emite cuando la pelota choca contra otro cuerpo.
signal rebote(pelota: Pelota, con: Node)

@export var velocidad_inicial: Vector2 = Vector2(260.0, 180.0)

var radio: float = 20.0

var _destello: float = 0.0

@onready var _forma: CollisionShape2D = $Forma


func _ready() -> void:
    radio = (_forma.shape as CircleShape2D).radius
    linear_velocity = velocidad_inicial
    # contact_monitor y max_contacts_reported están activados en pelota.tscn;
    # sin ellos el RigidBody2D no emite body_entered.
    body_entered.connect(_al_chocar)


func _process(delta: float) -> void:
    if _destello > 0.0:
        _destello = maxf(0.0, _destello - delta)
        queue_redraw()


func _al_chocar(cuerpo: Node) -> void:
    _destello = 0.15
    rebote.emit(self, cuerpo)
    queue_redraw()


func _draw() -> void:
    var color: Color = Color("ffb84d")
    if _destello > 0.0:
        color = Color("ffffff")
    draw_circle(Vector2.ZERO, radio, color)
    draw_arc(Vector2.ZERO, radio, 0.0, TAU, 32, Color("7a4b00"), 2.0)
```

Líneas	Qué hace
1–2	Clase global `Pelota`, hereda de `RigidBody2D`.
10–11	`signal rebote(pelota, con)`: se emite en cada choque; lleva la propia pelota y el cuerpo contra el que chocó.
13	`@export var velocidad_inicial`: valor por defecto (260, 180); en `laboratorio.tscn` cada pelota lo sobrescribe.
15	`radio = 20.0`: se corrige en `_ready`.
17	`_destello`: tiempo restante (s) del parpadeo blanco.
19	Referencia al hijo «Forma».
22–27	`_ready()`: lee el radio del `CircleShape2D` (línea 23); **asigna `linear_velocity`** con la velocidad inicial (24), que es lo único que «empuja» la pelota; conecta la señal `body_entered` del motor a `_al_chocar` (27). Esa señal solo existe porque `contact_monitor` y `max_contacts_reported` están activados en la escena.
30–33	`_process(delta)` (se llama cada cuadro de dibujo): si hay destello, lo reduce con `maxf(0.0, …)` para que no sea negativo y pide redibujar.
36–39	`_al_chocar(cuerpo)`: pone `_destello = 0.15`, **emite `rebote`** y redibuja. Se ejecuta cuando el motor avisa que la pelota entró en contacto con otro cuerpo (pared, obstáculo, jugador u otra pelota).
42–47	`_draw()`: círculo naranja (`ffb84d`), blanco si hay destello, y contorno marrón.

Frase para exponer: «La pelota no tiene código de rebote: el motor lo calcula con el material de rebote 1 y fricción 0. Nuestro código solo escucha el contacto y cuenta.»

8.3 `moneda.gd` — el Area2D

Idea: un `Area2D` **solo detecta**: no bloquea ni rebota. Avisa con señales cuando un cuerpo entra o sale de su zona.

SCRIPTS/MONEDA.GD · LÍNEAS 1–30

```gdscript
class_name Moneda
extends Area2D
## Moneda (Area2D): sirve para DETECTAR, no para bloquear.
##
## Un Area2D no choca físicamente; avisa con señales cuando un cuerpo
## entra o sale de su zona. Su collision_mask solo escanea la capa 2
## (jugador), así que las pelotas la atraviesan sin activarla.

## Se emite cuando el jugador entra en la zona de la moneda.
signal recogida(moneda: Moneda)

var radio: float = 16.0

@onready var _forma: CollisionShape2D = $Forma


func _ready() -> void:
    radio = (_forma.shape as CircleShape2D).radius
    body_entered.connect(_al_entrar_cuerpo)


func _al_entrar_cuerpo(cuerpo: Node2D) -> void:
    if cuerpo.is_in_group("jugador"):
        recogida.emit(self)
        queue_free()


func _draw() -> void:
    draw_circle(Vector2.ZERO, radio, Color("ffd93d"))
    draw_arc(Vector2.ZERO, radio, 0.0, TAU, 32, Color("8a6d00"), 2.0)
```

Líneas	Qué hace
10	`signal recogida(moneda)` : se emite cuando el jugador entra en la zona.
17–19	`_ready()` : lee el radio y conecta la señal `body_entered` del Area2D a `_al_entrar_cuerpo` .
22–25	`_al_entrar_cuerpo(cuerpo)` : si el cuerpo pertenece al grupo «jugador» (`is_in_group`), emite `recogida` y se elimina con `queue_free()` (se borra al final del cuadro).
28–30	`_draw()` : círculo amarillo con contorno.

Por qué las pelotas no la activan: la máscara de la moneda es 2 (solo escanea la capa del jugador); aunque la pelota pase por encima, esa capa no está en su máscara. Además, la comprobación de grupo en la línea 23 es una segunda protección.

8.4 `paredes.gd` , `obstaculo.gd` y `superposicion.gd` — dibujo

Los cuerpos de colisión **no se ven por sí solos**: Godot solo los muestra en modo depuración. Estos scripts dibujan las formas para que la escena se entienda.

SCRIPTS/PAREDES.GD · LÍNEAS 1–13

```
 1  extends StaticBody2D
 2  ## Paredes del escenario (StaticBody2D con varias CollisionShape2D).
 3  ##
 4  ## Un StaticBody2D participa en la detección pero el motor nunca lo mueve.
 5  ## Este script solo dibuja cada rectángulo hijo para que se vea el límite.
 6
 7
 8  func _draw() -> void:
 9      for hijo in get_children():
10          if hijo is CollisionShape2D:
11              var caja: RectangleShape2D = (hijo as CollisionShape2D).shape as RectangleShape2D
12              var esquina: Vector2 = (hijo as CollisionShape2D).position - caja.size / 2.0
13              draw_rect(Rect2(esquina, caja.size), Color("2b3350"))
```

`paredes.gd` : recorre los hijos (línea 9); si un hijo es `CollisionShape2D` (10), toma su `RectangleShape2D` (11), calcula la esquina superior izquierda restando medio tamaño a la posición (12) y dibuja el rectángulo de color `2b3350` (13). Así se ven las cuatro paredes con exactamente el tamaño que tiene la colisión.

SCRIPTS/OBSTACULO.GD · LÍNEAS 1–15

```
 1  extends StaticBody2D
 2  ## Obstáculo triangular (StaticBody2D con CollisionPolygon2D).
 3  ##
 4  ## Es un polígono convexo y está rotado: por eso su caja envolvente (AABB)
 5  ## es mayor que la forma real, y el motor necesita una prueba de fase estrecha
 6  ## precisa (el motor usa SAT para estas formas).
 7
 8  @onready var _forma: CollisionPolygon2D = $Forma
 9
10
11  func _draw() -> void:
12      draw_colored_polygon(_forma.polygon, Color("5b6b8c"))
13      var contorno: PackedVector2Array = _forma.polygon
14      contorno.append(contorno[0])
15      draw_polyline(contorno, Color.WHITE, 2.0)
```

`obstaculo.gd` : dibuja el polígono de la colisión (`_forma.polygon`) relleno de `5b6b8c` (línea 12) y luego un contorno blanco: copia los puntos, agrega el primero al final para cerrar la figura y llama a `draw_polyline` (13–15). Al estar rotado el nodo, el dibujo hereda la rotación.

SCRIPTS/SUPERPOSICION.GD · LÍNEAS 1–14

```
 1  extends Node2D
 2  ## Capa transparente dibujada por encima de todo el escenario.
 3  ## Pide a Laboratorio que dibuje sobre ella las cajas envolventes (AABB)
 4  ## y las líneas de contacto, para que queden visibles sobre los cuerpos.
 5
 6
 7  func _process(_delta: float) -> void:
 8      queue_redraw()
 9
10
11  func _draw() -> void:
12      var laboratorio: Node = get_parent()
13      if laboratorio.has_method("dibujar_depuracion"):
14          laboratorio.dibujar_depuracion(self)
```

`superposicion.gd` : `_process` pide redibujar cada cuadro (línea 7–8); `_draw` toma al padre (`Laboratorio`), comprueba que tenga el método `dibujar_depuracion` (13) y se lo llama pasándose a sí mismo como «lienzo» (14). Es un truco para que el laboratorio dibuje *encima* de todo: este nodo tiene `z_index = 10` .

8.5 `colision_manual.gd` — las pruebas de la fase estrecha escritas a mano

Idea: reproducir con el mismo procedimiento de la exposición las pruebas de colisión más simples, para poder compararlas con el motor y probarlas con números. Es una clase `RefCounted` con **funciones estáticas** (no hay que crear ningún objeto: se llama `ColisionManual.circulo_circulo(...)`).

Convención de resultado (en todas): devuelven un `Dictionary`. **Vacío `{}`** = no hay colisión. **`{"normal": Vector2, "profundidad": float}`** = sí hay colisión. La normal apunta del primer cuerpo hacia el segundo (en `circulo_aabb`, del círculo hacia la caja).

SCRIPTS/COLISION_MANUAL.GD · LÍNEAS 19–50

```
## Círculo contra círculo: hay colisión si |d|^2 < (ra + rb)^2.
static func circulo_circulo(ca: Vector2, ra: float, cb: Vector2, rb: float) -> Dictionary:
    var d: Vector2 = cb - ca
    var suma: float = ra + rb
    var dist2: float = d.length_squared()
    if dist2 >= suma * suma:
        return {}
    var dist: float = sqrt(dist2)
    var normal: Vector2 = Vector2.RIGHT
    if dist > 0.0:
        normal = d / dist
    return {"normal": normal, "profundidad": suma - dist}


## Caja contra caja (AABB): los intervalos deben solaparse en X y en Y.
## El eje con menor solape determina la normal y la profundidad.
static func aabb_aabb(a: Rect2, b: Rect2) -> Dictionary:
    var ox: float = minf(a.end.x, b.end.x) - maxf(a.position.x, b.position.x)
    var oy: float = minf(a.end.y, b.end.y) - maxf(a.position.y, b.position.y)
    if ox <= 0.0 or oy <= 0.0:
        return {}
    var centro_a: Vector2 = a.get_center()
    var centro_b: Vector2 = b.get_center()
    if ox < oy:
        var signo_x: float = 1.0
        if centro_b.x < centro_a.x:
            signo_x = -1.0
        return {"normal": Vector2(signo_x, 0.0), "profundidad": ox}
    var signo_y: float = 1.0
    if centro_b.y < centro_a.y:
        signo_y = -1.0
    return {"normal": Vector2(0.0, signo_y), "profundidad": oy}
```

Líneas	Qué hace
20	Firma de `circulo_circulo(ca, ra, cb, rb)`: centros y radios de los dos círculos.
21–23	`d = cb − ca`; `suma = ra + rb`; `dist2 = d.length_squared()`: se trabaja con la distancia **al cuadrado** para evitar la raíz cuadrada.
24–25	Si `dist2 >= suma²` no hay colisión: devuelve `{}`.
26–29	Calcula la distancia real `dist = sqrt(dist2)` (solo ahora, porque ya se sabe que chocan). Normal por defecto `Vector2.RIGHT` para el caso en que los centros coincidan (evita dividir entre 0); si no, `normal = d / dist`.
30	Devuelve normal y `profundidad = suma − dist`.
35	`aabb_aabb(a, b)` recibe dos `Rect2`.
36–37	Solape en cada eje: `ox = min(fin_a, fin_b) − max(inicio_a, inicio_b)`, igual para `oy`. Si el resultado es positivo hay solape en ese eje.
38–39	Si `ox ≤ 0` **o** `oy ≤ 0`, no hay colisión: `{}`.
40–50	Si `ox < oy`, el eje de menor solape es X: la normal es (±1, 0), con signo + si el centro de B está a la derecha del de A; profundidad `ox`. Si no, el eje es Y: normal (0, ±1) y profundidad `oy`.

SCRIPTS/COLISION_MANUAL.GD · LÍNEAS 56–83

```
56  static func circulo_aabb(centro: Vector2, radio: float, caja: Rect2) -> Dictionary:
57      var p: Vector2 = Vector2(
58          clampf(centro.x, caja.position.x, caja.end.x),
59          clampf(centro.y, caja.position.y, caja.end.y)
60      )
61      var d: Vector2 = centro - p
62      var dist2: float = d.length_squared()
63      if dist2 == 0.0:
64          # El centro está dentro de la caja: se sale por la cara más cercana.
65          var izq: float = centro.x - caja.position.x
66          var der: float = caja.end.x - centro.x
67          var arr: float = centro.y - caja.position.y
68          var aba: float = caja.end.y - centro.y
69          var menor: float = minf(minf(izq, der), minf(arr, aba))
70          var normal: Vector2 = Vector2.DOWN
71          if menor == izq:
72              normal = Vector2.RIGHT
73          elif menor == der:
74              normal = Vector2.LEFT
75          elif menor == arr:
76              normal = Vector2.DOWN
77          else:
78              normal = Vector2.UP
79          return {"normal": normal, "profundidad": menor + radio}
80      if dist2 >= radio * radio:
81          return {}
82      var dist: float = sqrt(dist2)
83      return {"normal": -d / dist, "profundidad": radio - dist}
```

Líneas	Qué hace
56	`circulo_aabb(centro, radio, caja)`.
57–60	**Punto P** de la caja más cercano al centro: se recorta (`clampf`) cada coordenada del centro al rango de la caja.
61–62	`d = centro − P` y su distancia al cuadrado.
63–79	**Caso especial:** si `dist2 == 0` el centro está *dentro* de la caja. Se calcula la distancia del centro a cada una de las cuatro caras (izq, der, arr, aba), se toma la menor y la normal apunta hacia afuera por esa cara; `profundidad = menor + radio`.
80–81	Si `dist2 >= radio²` : sin colisión, `{}`.
82–83	Colisión: normal `−d / dist` (del círculo hacia la caja) y `profundidad = radio − dist`.

SCRIPTS/COLISION_MANUAL.GD · LÍNEAS 86–125

```
 86  ## Teorema del eje separador (SAT) para dos polígonos convexos.
 87  ## Se prueban las normales de las aristas de ambos; si en algún eje las
 88  ## proyecciones no se solapan, no hay colisión (salida temprana).
 89  static func sat_poligonos(va: PackedVector2Array, vb: PackedVector2Array) -> Dictionary:
 90      var menor: float = INF
 91      var eje_menor: Vector2 = Vector2.ZERO
 92      for poligono in [va, vb]:
 93          var n: int = poligono.size()
 94          for i in n:
 95              var arista: Vector2 = poligono[(i + 1) % n] - poligono[i]
 96              var eje: Vector2 = Vector2(-arista.y, arista.x).normalized()
 97              var pa: Vector2 = _proyectar(va, eje)
 98              var pb: Vector2 = _proyectar(vb, eje)
 99              var solape: float = minf(pa.y, pb.y) - maxf(pa.x, pb.x)
100              if solape <= 0.0:
101                  return {}
102              if solape < menor:
103                  menor = solape
104                  eje_menor = eje
105      if (_centro(vb) - _centro(va)).dot(eje_menor) < 0.0:
106          eje_menor = -eje_menor
107      return {"normal": eje_menor, "profundidad": menor}
108
109
110  ## Proyecta los vértices sobre un eje unitario. Devuelve Vector2(mínimo, máximo).
111  static func _proyectar(vertices: PackedVector2Array, eje: Vector2) -> Vector2:
112      var minimo: float = INF
113      var maximo: float = -INF
114      for v in vertices:
115          var valor: float = v.dot(eje)
116          minimo = minf(minimo, valor)
117          maximo = maxf(maximo, valor)
118      return Vector2(minimo, maximo)
119
120
121  static func _centro(vertices: PackedVector2Array) -> Vector2:
122      var suma: Vector2 = Vector2.ZERO
123      for v in vertices:
124          suma += v
125      return suma / float(vertices.size())
```

Líneas	Qué hace
89–91	`sat_poligonos(va, vb)` : recibe los vértices de dos polígonos convexos. `menor` guarda el menor solape visto (empieza en infinito) y `eje_menor` su eje.
92–94	Recorre **las aristas de ambos polígonos** (`for poligono in [va, vb]`), con índice `i` y el siguiente vértice `(i + 1) % n` .
95–96	`arista` = vector entre dos vértices consecutivos; `eje` = perpendicular `(−arista.y, arista.x)` normalizada.
97–99	Proyecta ambos polígonos sobre ese eje con `_proyectar` (devuelve mín y máx) y calcula el solape: `min(máx_a, máx_b) − max(mín_a, mín_b)` .
100–101	**Salida temprana:** si el solape es ≤ 0, ese eje separa a los polígonos: no hay colisión, devuelve `{}` .
102–104	Si el solape es el menor hasta ahora, lo guarda con su eje.
105–106	Orienta la normal de A hacia B: si el vector entre centros apunta contra el eje, se invierte.
107	Devuelve el eje de menor solape como normal y ese solape como profundidad (el MTV).
111–118	`_proyectar(vertices, eje)` : para cada vértice calcula el producto punto con el eje (su «sombra») y devuelve `Vector2(mínimo, máximo)` .
121–125	`_centro(vertices)` : promedio de los vértices.

Límite de SAT en este código: solo vale para polígonos **convexos**, como en la teoría. Los vértices deben recibirse en coordenadas globales y ya rotados; en el laboratorio **no se usa** `sat_poligonos` (solo se prueba en `pruebas_manuales.gd`); el laboratorio usa `circulo_aabb` .

8.6 `laboratorio.gd` — el cerebro de la escena

Idea: reunir los eventos de todos los objetos, contarlos, dibujar la depuración y conectar con la teoría (AABB, grafos, fase estrecha). **No recalcula colisiones**: solo escucha las señales y consulta al motor.

SCRIPTS/LABORATORIO.GD · LÍNEAS 1–58

```
class_name Laboratorio
extends Node2D
## Escena principal: reúne los cuerpos y muestra qué ocurre en cada colisión.
##
## Hace cuatro cosas:
##  1. Cuenta los eventos que emiten los nodos (choques, rebotes, monedas).
##  2. Dibuja la caja envolvente (AABB) de cada cuerpo (tecla Espacio).
##  3. Agrupa los cuerpos que se tocan con un recorrido en profundidad (DFS),
##     tratando cada cuerpo como un vértice y cada contacto como una arista.
##  4. Compara la prueba manual círculo-AABB con lo que reporta el motor.

## Margen (px) que se suma al radio en la prueba manual: el motor separa los
## cuerpos al resolver el contacto, así que casi nunca quedan solapados.
const MARGEN_CONTACTO: float = 2.0

var _choques_jugador: int = 0
var _rebotes: int = 0
var _monedas_recogidas: int = 0
var _monedas_total: int = 0
var _mostrar_aabb: bool = true
var _grupos: int = 0
var _manual_pelota1: bool = false
var _motor_pelota1: bool = false

@onready var _jugador: Jugador = $Jugador
@onready var _texto: Label = $Interfaz/Texto


func _ready() -> void:
    _jugador.choque.connect(_al_chocar_jugador)
    for pelota: Pelota in get_tree().get_nodes_in_group("pelotas"):
        pelota.rebote.connect(_al_rebotar)
    for moneda: Moneda in $Monedas.get_children():
        moneda.recogida.connect(_al_recoger)
        _monedas_total += 1
    _actualizar_texto()


func _unhandled_input(evento: InputEvent) -> void:
    if evento.is_action_pressed("ui_accept"):
        _mostrar_aabb = not _mostrar_aabb
    elif evento.is_action_pressed("ui_cancel"):
        get_tree().reload_current_scene()


func _physics_process(_delta: float) -> void:
    _grupos = _contar_grupos()
    var pelotas: Array = get_tree().get_nodes_in_group("pelotas")
    if not pelotas.is_empty():
        var primera: Pelota = pelotas[0]
        # Prueba manual (explicada en la exposición) frente al resultado del motor.
        var manual: Dictionary = ColisionManual.circulo_aabb(
            primera.global_position, primera.radio + MARGEN_CONTACTO, _jugador.caja_global()
        )
        _manual_pelota1 = not manual.is_empty()
        _motor_pelota1 = primera.get_colliding_bodies().has(_jugador)
    _actualizar_texto()

```

Líneas	Qué hace
1–2	Clase global `Laboratorio` que extiende `Node2D`.
14	`MARGEN_CONTACTO = 2.0`: píxeles que se suman al radio en la prueba manual. Motivo: el motor **separa** los cuerpos al resolver el choque, así que casi nunca quedan realmente solapados; con el margen la prueba manual detecta «casi tocando».
16–23	Variables de estado: contadores (`_choques_jugador`, `_rebotes`, `_monedas_recogidas`, `_monedas_total`), `_mostrar_aabb` (empieza en true), `_grupos` y los dos resultados de la comparación (`_manual_pelota1`, `_motor_pelota1`).
25–26	`@onready`: referencias al jugador (`$Jugador`) y al Label (`$Interfaz/Texto`).
29–36	`_ready()`: **conecta las señales**: `choque` del jugador; `rebote` de cada pelota del grupo «pelotas»; `recogida` de cada moneda de `$Monedas` (y cuenta el total). Termina actualizando el texto.
39–43	`_unhandled_input`: `ui_accept` (Espacio o Enter) alterna `_mostrar_aabb`; `ui_cancel` (Esc) recarga la escena (`reload_current_scene`).
46–57	`_physics_process`, cada paso de física: (47) recalcula los grupos; (48–50) toma la **primera pelota** del grupo; (52–54) ejecuta `ColisionManual.circulo_aabb` con el centro de la pelota, su radio + margen y la AABB del jugador; (55) guarda si la prueba manual dio colisión; (56) pregunta al motor si esa pelota está en contacto con el jugador (`get_colliding_bodies().has(_jugador)`); (57) actualiza el texto.

SCRIPTS/LABORATORIO.GD · LÍNEAS 60–111

```
60  ## Cuerpos que participan en el grafo de contactos: el jugador y las pelotas.
61  func _cuerpos_del_grafo() -> Array:
62      var cuerpos: Array = [_jugador]
63      cuerpos.append_array(get_tree().get_nodes_in_group("pelotas"))
64      return cuerpos
65
66
67  ## Vecinos de un cuerpo = cuerpos del grafo con los que está en contacto.
68  func _vecinos(cuerpo: Node) -> Array:
69      var resultado: Array = []
70      if cuerpo is RigidBody2D:
71          for otro in (cuerpo as RigidBody2D).get_colliding_bodies():
72              if otro is RigidBody2D or otro is Jugador:
73                  resultado.append(otro)
74      elif cuerpo is Jugador:
75          for pelota in get_tree().get_nodes_in_group("pelotas"):
76              if (pelota as RigidBody2D).get_colliding_bodies().has(cuerpo):
77                  resultado.append(pelota)
78      return resultado
79
80
81  ## Componentes conexas del grafo de contactos (recorrido DFS con pila).
82  func _contar_grupos() -> int:
83      var visitados: Dictionary = {}
84      var grupos: int = 0
85      for inicio in _cuerpos_del_grafo():
86          if visitados.has(inicio):
87              continue
88          grupos += 1
89          var pila: Array = [inicio]
90          while not pila.is_empty():
91              var actual: Node = pila.pop_back()
92              if visitados.has(actual):
93                  continue
94              visitados[actual] = true
95              for vecino in _vecinos(actual):
96                  if not visitados.has(vecino):
97                      pila.append(vecino)
98      return grupos
99
100
101 func _al_chocar_jugador(_con: Node, _normal: Vector2) -> void:
102     _choques_jugador += 1
103
104
105 func _al_rebotar(_pelota: Pelota, _con: Node) -> void:
106     _rebotes += 1
107
108
109 func _al_recoger(_moneda: Moneda) -> void:
110     _monedas_recogidas += 1
111
```

Líneas	Qué hace
61–64	`_cuerpos_del_grafo()`: los **vértices** del grafo: el jugador y todas las pelotas.
68–78	`_vecinos(cuerpo)`: las **aristas**. Si el cuerpo es una pelota, sus vecinos son los cuerpos de `get_colliding_bodies()` que sean pelotas o el jugador (se ignoran paredes y obstáculo). Si es el jugador, sus vecinos son las pelotas cuyo `get_colliding_bodies()` lo incluye (el jugador no tiene `contact_monitor`, por eso se pregunta desde las pelotas).
82–98	`_contar_grupos()`: **recorrido en profundidad (DFS) iterativo con pila**. `visitados` es un Dictionary; para cada vértice no visitado se suma un grupo (88) y se explora con una pila (89–97): se saca un vértice (`pop_back`), si ya fue visitado se salta, se marca y se apilan sus vecinos no visitados. El resultado es el número de **componentes conexas**: grupos de cuerpos que se tocan entre sí (directa o indirectamente). Con 4 cuerpos separados da 4.
101–110	Manejadores de señal: cada uno suma 1 a su contador. Los parámetros llevan guion bajo (`_con`, `_normal`, …) porque no se usan.

Costo del DFS: O(V + E), con V = 4 vértices (jugador + 3 pelotas) y E ≤ el número de contactos. Es el mismo recorrido visto en clase en el tema de Grafos.

SCRIPTS/LABORATORIO.GD · LÍNEAS 113–144

```
113  func _actualizar_texto() -> void:
114      var si_no_manual: String = "sí" if _manual_pelota1 else "no"
115      var si_no_motor: String = "sí" if _motor_pelota1 else "no"
116      _texto.text = (
117          "LABORATORIO DE COLISIONES\n"
118          + "Mover: flechas o WASD  |  Espacio: ver AABB  |  Esc: reiniciar\n\n"
119          + "Choques del jugador (CharacterBody2D): %d\n" % _choques_jugador
120          + "Rebotes de pelotas (RigidBody2D): %d\n" % _rebotes
121          + "Monedas recogidas (Area2D): %d de %d\n" % [_monedas_recogidas, _monedas_total]
122          + "Grupos de cuerpos en contacto (DFS): %d\n" % _grupos
123          + "Pelota 1 vs jugador -> prueba manual: %s | motor: %s" % [si_no_manual, si_no_motor]
124      )
125
126
127  ## Llamado por Superposicion: dibuja AABB y líneas de contacto.
128  func dibujar_depuracion(lienzo: CanvasItem) -> void:
129      if not _mostrar_aabb:
130          return
131      var caja_j: Rect2 = _jugador.caja_global()
132      lienzo.draw_rect(caja_j, Color("00e5ff"), false, 2.0)
133      for pelota: Pelota in get_tree().get_nodes_in_group("pelotas"):
134          var mitad: Vector2 = Vector2(pelota.radio, pelota.radio)
135          var caja_p: Rect2 = Rect2(pelota.global_position - mitad, mitad * 2.0)
136          lienzo.draw_rect(caja_p, Color("00e5ff"), false, 2.0)
137      # Aristas del grafo de contactos.
138      for cuerpo in _cuerpos_del_grafo():
139          for vecino in _vecinos(cuerpo):
140              lienzo.draw_line(
141                  (cuerpo as Node2D).global_position,
142                  (vecino as Node2D).global_position,
143                  Color("ff4d9a"), 3.0
144              )
```

Líneas	Qué hace
113–124	`_actualizar_texto()` : convierte los booleanos en «sí»/«no» y arma el texto del panel concatenando líneas con `+` y el operador `%` de formato (`%d` para enteros). Se llama cada cuadro de física y al iniciar.
128–131	`dibujar_depuracion(lienzo)` : lo llama `Superposicion` . Si `_mostrar_aabb` es falso, no dibuja nada. Dibuja la AABB del jugador en cian (`00e5ff` , 2 px).
133–136	Para cada pelota calcula su AABB: la caja cuadrada de lado 2×radio centrada en la pelota (`mitad = Vector2(radio, radio)`) y la dibuja en cian.
137–144	**Aristas del grafo:** para cada cuerpo y cada vecino dibuja una línea rosa (`ff4d9a` , 3 px) entre sus centros.

Detalles finos que pueden preguntar. (1) Solo se dibujan AABB del jugador y de las pelotas; **no** del obstáculo ni de las paredes. (2) La AABB de una pelota es el cuadrado que circunscribe su círculo; la del jugador coincide con su forma porque ya es un rectángulo alineado con los ejes. (3) «Pelota 1» es la primera del grupo «pelotas» en el orden del árbol. (4) `get_colliding_bodies()` existe porque el RigidBody2D tiene `contact_monitor` activado.

8.7 `pruebas_manuales.gd` — comprobaciones sin abrir el editor

Idea: un script que **extiende** `SceneTree` y se ejecuta desde la consola de Godot, sin cargar ninguna escena. Corre 17 comprobaciones y sale con código 0 (todo bien) o 1 (hay fallos). Se explica caso por caso en la sección 10.

SCRIPTS/PRUEBAS_MANUALES.GD · LÍNEAS 1–22

```
 1  extends SceneTree
 2  ## Pruebas de las funciones de ColisionManual, sin abrir el editor ni ninguna escena.
 3  ##
 4  ## Ejecutar desde la carpeta del proyecto:
 5  ##   godot --headless --path . --script res://scripts/pruebas_manuales.gd
 6  ## (el ejecutable puede llamarse godot, godot4 o Godot_v4.x_win64_console.exe)
 7  ##
 8  ## Cada caso es un cálculo hecho a mano; los mismos números aparecen como
 9  ## trazas en la presentación. Código de salida: 0 = todo bien, 1 = hay fallos.
10
11  const Manual := preload("res://scripts/colision_manual.gd")
12
13  var _total: int = 0
14  var _fallos: int = 0
15
16
17  func _init() -> void:
18      var caja_a: Rect2 = Rect2(-10, -10, 20, 20)
19
20      # Círculo - círculo
21      var r1: Dictionary = Manual.circulo_circulo(Vector2(0, 0), 10.0, Vector2(15, 0), 10.0)
22      _verificar("circulo-circulo: chocan", not r1.is_empty())
```

Líneas	Qué hace
1	`extends SceneTree`: al ejecutarse con `--script`, Godot lo usa como bucle principal en lugar de una escena.
11	`const Manual := preload("res://scripts/colision_manual.gd")`: carga el script de pruebas manuales como constante (en este modo no se garantiza que los nombres de clase globales estén registrados, por eso se usa `preload`).
13–14	Contadores `_total` y `_fallos`.
17–18	`_init()` es el punto de entrada. Define la caja A = Rect2(−10, −10, 20, 20) (de −10 a 10 en ambos ejes).
70–71	Al final imprime «Pruebas: %d | fallos: %d» y llama a `quit(1 si hay fallos, 0 si no)`.
74–78	`_rombo(centro, radio)`: arma un rombo de 4 vértices a ±radio del centro.
81–87	`_verificar(nombre, condicion)`: suma al total; imprime «ok» o «FALLO» con el nombre y cuenta el fallo.
90–95	`_cerca` y `_cerca_v`: comparan con tolerancia de 0,001 (números decimales no se comparan con `==`).

9. Qué pasa en cada cuadro (flujo completo y mapa de señales)

9.1 Al iniciar

- Godot carga `laboratorio.tscn` (escena principal) e instancia todas las escenas hijas.
- Cada nodo ejecuta su `_ready()`: Jugador y Moneda leen su forma; **Pelota** fija `linear_velocity` y conecta `body_entered`; **Moneda** conecta `body_entered`.
- `Laboratorio._ready()` conecta las señales de jugador, pelotas y monedas y pinta el texto inicial (todo en 0).

9.2 En cada paso de física (60 por segundo)

Orden	Quién	Qué ocurre
1	Motor de física	Mueve los `RigidBody2D` (pelotas) según su velocidad; **detecta** contactos (fase amplia + estrecha internas) y calcula el rebote con el PhysicsMaterial.
2	Pelota (por señal del motor)	Si hubo contacto nuevo: `body_entered` → `_al_chocar` → emite `rebote` → `Laboratorio._al_rebotar` suma 1.
3	`Jugador._physics_process`	Lee teclado, asigna `velocity`, `move_and_slide()`; si empezó un contacto emite `choque` → `Laboratorio._al_chocar_jugador` suma 1.
4	Moneda (por señal del motor)	Si el jugador entró: `body_entered` → emite `recogida` → `Laboratorio._al_recoger` suma 1 → la moneda se elimina.
5	`Laboratorio._physics_process`	Recalcula los grupos (DFS), hace la comparación manual/motor y actualiza el texto.

El orden exacto entre los `_physics_process` de distintos nodos lo decide Godot según el árbol; para entender el programa basta con saber que todo ocurre dentro del mismo paso de física.

9.3 En cada cuadro de dibujo

- `Superposicion._process` pide redibujar → `_draw` llama a `Laboratorio.dibujar_depuracion` → se pintan las AABB (cian) y las aristas del grafo (rosa) si `_mostrar_aabb` es verdadero.
- `Pelota._process` reduce el destello y redibuja si hace falta; `Jugador` redibuja tras moverse.

9.4 Mapa de señales

Señal	Quién la emite	Cuándo	Quién la escucha	Efecto
`body_entered` (de Godot)	`Pelota` (RigidBody2D)	La pelota entra en contacto con un cuerpo	`Pelota._al_chocar`	Destello y emite `rebote`
`rebote`	`Pelota`	Cada choque	`Laboratorio._al_rebotar`	`_rebotes += 1`
`choque`	`Jugador`	Empieza un contacto en `move_and_slide`	`Laboratorio._al_chocar_jugador`	`_choques_jugador += 1`
`body_entered` (de Godot)	`Moneda` (Area2D)	Un cuerpo entra en la zona	`Moneda._al_entrar_cuerpo`	Si es el jugador: emite `recogida` y se elimina
`recogida`	`Moneda`	El jugador la toca	`Laboratorio._al_recoger`	`_monedas_recogidas += 1`

Por qué señales: el objeto avisa que algo ocurrió sin saber quién escucha; el laboratorio cuenta sin que los objetos conozcan el panel. Eso mantiene cada script con una sola responsabilidad.

10. Las 17 pruebas manuales, caso por caso

Caja A = Rect2(−10, −10, 20, 20): ocupa de −10 a 10 en X y en Y. Tolerancia de comparación: 0,001.

#	Prueba	Entrada	Cálculo a mano	Esperado
1–3	Círculo–círculo: chocan, profundidad, normal	Centros (0,0) y (15,0), radios 10 y 10	d = 15; suma de radios = 20; 15 < 20 → colisión; profundidad = 20 − 15	Hay colisión; profundidad 5; normal (1, 0)
4	Círculo–círculo: separados	Centros (0,0) y (25,0), radios 10	$dist^2 = 625 \geq 400$ ($suma^2$)	Vacío (sin colisión)
5–6	AABB–AABB: profundidad, normal	A y B = Rect2(5, −5, 20, 20)	ox = min(10, 25) − max(−10, 5) = 5; oy = min(10, 15) − max(−10, −5) = 15; ox < oy → eje X; B a la derecha → +1	Profundidad 5; normal (1, 0)
7	AABB–AABB: separadas	B = Rect2(20, −10, 20, 20)	ox = min(10, 40) − max(−10, 20) = −10 ≤ 0	Vacío
8–9	Círculo–AABB: profundidad, normal	Centro (17, 0), radio 8	P = (10, 0); d = (7, 0); 49 < 64; dist = 7; profundidad = 8 − 7; normal = −d/dist	Profundidad 1; normal (−1, 0)
10–11	Círculo–AABB: centro dentro	Centro (5, 0), radio 8	P = centro → $dist^2 = 0$; distancias a caras: izq 15, der 5, arr 10, aba 10; menor = 5 (derecha); normal hacia la izquierda; profundidad = 5 + 8	Profundidad 13; normal (−1, 0)
12	Círculo–AABB: separados	Centro (30, 0), radio 8	P = (10, 0); $dist^2 = 400 \geq 64$	Vacío
13–14	SAT rombos: profundidad y normal diagonal	Rombos de radio 10 con centros (0,0) y (15,0)	En el eje diagonal (±0,707; ±0,707): A proyecta en [−7,07; 7,07], B en [−17,68; −3,54]; solape = −3,54 − (−7,07) = 3,54 = 5/√2	Profundidad 3,5355; |normal.x| = 0,70711 y normal.x > 0
15	SAT rombos: separados	Centro de B en (25, 0)	En el eje diagonal el solape es −3,54 ≤ 0	Vacío
16–17	SAT cuadrados: profundidad y normal	A = cuadrado $[-10,10]^2$; B = cuadrado con x de 5 a 25 e y de −5 a 15	Mismo caso que AABB: el eje X da solape 5 (el menor); la normal se orienta hacia B	Profundidad 5; normal (1, 0)

Los mismos números aparecen como trazas en las diapositivas 7 y 8 (círculos a distancia 15 con radio 10 → profundidad 5; cajas con ox = 5 y oy = 15; rombos con solape 3,54).

11. Cómo ejecutarlo y qué comandos usar

En el editor: abrir Godot 4.7.2 → Importar → elegir `project.godot` de la carpeta → esperar la importación → pulsar **F5** (reproducir proyecto). La escena principal ya está configurada.

Desde la terminal, en la carpeta del proyecto, con el ejecutable de consola de Godot (en el equipo del grupo está en `C:\Users\isaac\OneDrive\Documents\EJECUTABLE GODOT\Godot_v4.7.2-stable_win64_console.exe`; si no está en el PATH, usar la ruta completa y entre comillas):

```
:: 1) Pruebas manuales (17 comprobaciones, sin abrir ventana)
"…\Godot_v4.7.2-stable_win64_console.exe" --headless --path . --script res://scripts/pruebas_manuales.gd

:: 2) Comprobar que el proyecto carga sin errores
"…\Godot_v4.7.2-stable_win64_console.exe" --headless --path . --quit

:: 3) Grabar cuadros de la escena corriendo (sin teclado)
"…\Godot_v4.7.2-stable_win64_console.exe" --path . --write-movie capturas\cuadro.png --fixed-fps 30 --
quit-after 300
```

- `--headless`: sin ventana ni gráficos (sirve para pruebas).
- `--path .`: carpeta del proyecto.
- `--script`: ejecuta ese script como bucle principal.
- `--quit`: sale tras cargar.
- `--write-movie`: guarda cada cuadro como PNG; `--fixed-fps 30` fija 30 cuadros por segundo; `--quit-after 300` sale a los 300 cuadros.

12. Guion de la demostración en vivo (unos 2:30)

Antes de empezar: tener Godot abierto con el proyecto y una terminal lista en la carpeta. Quien demuestra explica mientras otro integrante maneja, o lo hace uno solo con calma. Si algo falla, usar la captura de la diapositiva 18.

Paso	Qué hacer	Qué debería verse	Qué decir
1	Pulsar F5.	Se abre la escena; las pelotas ya se mueven; contadores en 0 (los rebotes empiezan a subir solos).	«Esta es la escena: cuatro tipos de cuerpo. Las pelotas son RigidBody2D, las moverá el motor.»
2	Mover el cuadrado azul contra una pared con las flechas.	Se detiene y se desliza; se pone rojo un instante; sube «Choques del jugador».	«El jugador es un CharacterBody2D: Godot detecta, pero la respuesta la programamos con move_and_slide.»
3	Esperar o acercarse a que una pelota golpee al jugador.	La pelota rebota y parpadea en blanco; sube «Rebotes de pelotas».	«Aquí el motor calcula el rebote; nuestro código solo escucha la señal y cuenta.»
4	Caminar hasta una moneda amarilla.	La moneda desaparece; «Monedas recogidas» pasa a 1 de 3.	«La moneda es un Area2D: solo detecta. La máscara solo ve al jugador, por eso las pelotas la atraviesan.»
5	Pulsar Espacio.	Aparecen/desaparecen los rectángulos cian (AABB) y las líneas rosa entre cuerpos en contacto.	«Estas son las cajas envolventes: el filtro barato de la fase amplia. Las líneas son el grafo de contactos; contamos los grupos con DFS.»
6	Señalar la última línea del panel.	«prueba manual: sí/no | motor: sí/no» para Pelota 1 y el jugador.	«Comparamos nuestra prueba círculo–AABB con lo que dice el motor.»
7	En la terminal, ejecutar las pruebas manuales (comando 1).	Imprime las 17 líneas «ok» y «Pruebas: 17 | fallos: 0».	«Estas pruebas son cálculos hechos a mano, los mismos de las diapositivas 7 y 8.»

Si algo sale distinto: Esc reinicia la escena. Si Godot no abre, mostrar la captura de la diapositiva 18 y ejecutar solo el comando de pruebas. Decir la verdad sobre lo que ocurra: el criterio de la rúbrica premia explicar el código, no que todo salga perfecto.

13. Qué está verificado y qué no

Comprobación	Estado	Cómo se hizo / evidencia
Sintaxis de los scripts	Verificado	Análisis con gdtoolkit (gdparse y gdlint) sin errores.
Lógica de `colision_manual.gd`	Verificado	Traducción a Python: 17 de 17 correctos; y ejecución en Godot (abajo).
17 pruebas en Godot	Verificado	`--headless --script res://scripts/pruebas_manuales.gd` en Godot 4.7.2-stable: 17 pruebas, 0 fallos, salida 0 (informado por Claude Code en el equipo del grupo).
El proyecto carga sin errores	Verificado	`--headless --path . --quit`: sin errores ni advertencias, salida 0.
La escena corre; pelotas, rebotes, contadores y AABB	Verificado en grabación	Cuadros PNG de `--write-movie`: pelotas moviéndose; rebotes 2, 11, 26; choques del jugador 1, 2, 4; AABB cian; «Grupos (DFS): 4».
Movimiento del jugador con teclado	**No verificado**	La grabación no usa teclado. Probar con F5.
Recogida de monedas	**No verificado**	Siempre «0 de 3» en la grabación. Probar con F5.
Tecla Espacio y Esc	**No verificado**	Depende de entrada de teclado.
Coincidencia manual/motor en el panel	**No verificado**	En los cuadros vistos ambos dijeron «no»; no se observó un caso de contacto simultáneo.
Efecto de `linear_damp_mode` = Replace	No verificado	Tomado de la documentación; en la grabación las pelotas parecen conservar velocidad, pero no se midió.

Observación sin explicar del todo: en la grabación el cuadrado azul se desplazó unos 22 px (de x≈157 a x≈135) sin que nadie pulsara teclas, después de recibir golpes de las pelotas. Probablemente es la corrección de penetración del motor, pero no se investigó. Si preguntan, decirlo así.

14. Límites de la implementación

- Es un **laboratorio didáctico**, no un juego: no tiene objetivo ni fin.
- La fase amplia que se ve (AABB) es solo **visual**: no se usa para decidir colisiones; las decisiones las toma el motor.
- `sat_poligonos` y `aabb_aabb` solo se prueban en `pruebas_manuales.gd`; el laboratorio solo usa `circulo_aabb` en la comparación.
- El grafo de contactos solo incluye al jugador y a las pelotas, no a paredes ni obstáculo.
- No hay detección continua: una pelota muy rápida podría atravesar una pared (con estas velocidades y paredes de 20 px no se espera).
- El conteo de rebotes duplica los choques pelota–pelota (cada pelota cuenta el suyo).
- No hay pruebas automáticas de la escena, solo de las funciones matemáticas.
- Mejoras posibles: usar una cuadrícula propia como fase amplia y comparar tiempos; añadir QuadTree; activar `continuous_cd` y probar con una pelota rápida; ejecutar la comparación manual/motor para todas las pelotas.

15. Preguntas probables sobre el código

1. ¿Qué hace exactamente su implementación?
Es un laboratorio 2D en Godot con los cuatro tipos de cuerpo de colisión (Area2D, StaticBody2D, RigidBody2D, CharacterBody2D), un panel con contadores, cajas AABB dibujadas, un grafo de contactos recorrido con DFS y una versión manual de las pruebas de la fase estrecha.

2. ¿Ustedes programaron la detección de colisiones?
La detección real la hace Godot. Nosotros programamos las **respuestas** del jugador (move_and_slide y señal), la lógica de eventos, el grafo de contactos y una versión manual de las pruebas de fase estrecha para compararla con el motor y probarla con números.

3. ¿Por qué el jugador es CharacterBody2D y la pelota RigidBody2D?
El jugador lo controlamos con el teclado y queremos decidir cómo responde (deslizar), así que usamos CharacterBody2D. La pelota queremos que rebote con física, así que dejamos que el motor calcule la respuesta con RigidBody2D.

4. ¿Por qué la moneda es Area2D?
Porque solo debe detectar al jugador sin bloquearlo. Un Area2D no choca físicamente; avisa con señales.

5. ¿Qué son collision_layer y collision_mask y cómo las usaron?
Layer = en qué capa está el objeto; mask = qué capas escanea. Usamos cuatro capas con nombre. El jugador está en la 2 y escanea 1 y 3 (valor 5); la pelota está en la 3 y escanea 1, 2 y 3 (valor 7); la moneda está en la 4 y solo escanea la 2.

6. ¿Por qué la pelota guarda collision_layer = 4 si está en la capa 3?
Porque se guarda la máscara de bits: capa 1 = 1, capa 2 = 2, capa 3 = 4, capa 4 = 8.

7. ¿Para qué sirven contact_monitor y max_contacts_reported?
Sin ellos el RigidBody2D no emite body_entered ni permite get_colliding_bodies(). Los activamos en pelota.tscn (true y 8).

8. ¿Dónde está el código del rebote?
No hay: lo calcula el motor con el PhysicsMaterial (bounce 1, friction 0). Nosotros solo asignamos linear_velocity inicial.

9. ¿Qué hace move_and_slide?
Mueve el CharacterBody2D con su velocity y, si choca, lo desliza a lo largo de la superficie en vez de detenerlo.

10. ¿Por qué usan señales?
Para desacoplar: pelota, moneda y jugador avisan eventos sin conocer al laboratorio ni al panel. Cada script tiene una sola responsabilidad.

11. ¿Cómo funciona el grafo de contactos?
Cada cuerpo (jugador y pelotas) es un vértice; hay arista si dos están en contacto (get_colliding_bodies). Con un DFS iterativo con pila contamos componentes conexas: los grupos de cuerpos que se tocan. Con cuatro cuerpos separados da 4.

12. ¿Qué relación tiene con lo visto en clase?
Grafos (DFS y componentes conexas) está implementado. Árboles, ordenamiento y hash se relacionan con QuadTree, sweep and prune y cuadrícula, que explicamos como teoría; Godot ya trae su propia fase amplia.

13. ¿Por qué comparan la prueba manual con el motor y qué es MARGEN_CONTACTO?
Para mostrar que nuestra prueba círculo–AABB coincide con lo que reporta el motor. Como el motor separa los cuerpos al resolver el choque, casi nunca se solapan; sumamos 2 px al radio para detectar «casi tocando».

14. ¿Cómo funciona circulo_aabb?
Se busca el punto P de la caja más cercano al centro recortando (clamp) sus coordenadas; si la distancia del centro a P es menor que el radio hay colisión; la profundidad es radio − distancia y la normal va del círculo a la caja. Si el centro está dentro de la caja se sale por la cara más cercana.

15. ¿Por qué trabajan con distancias al cuadrado?
Para evitar la raíz cuadrada, que es más costosa; solo se calcula cuando ya se sabe que hay colisión.

16. ¿Cómo funciona SAT en su código?
Para cada arista de ambos polígonos toma la perpendicular como eje, proyecta los dos polígonos y calcula el solape; si algún eje no solapa, no hay colisión (salida temprana); si todos solapan, el menor solape es la profundidad y su eje la normal.

17. ¿Cómo comprobaron que funciona?
17 pruebas numéricas hechas a mano, ejecutadas en Godot 4.7.2 sin fallos; el proyecto cargó sin errores; grabamos la escena corriendo y vimos los contadores y las AABB. Falta confirmar con teclado el movimiento del jugador y las monedas.

18. ¿Qué algoritmos usa Godot internamente?
Según el código fuente consultado (rama master, que puede diferir de la 4.7), una fase amplia con BVH y una fase estrecha con SAT. No lo comprobamos ejecutando; es lectura de código.

19. ¿Qué mejorarían?
Una fase amplia propia (cuadrícula o QuadTree) para medir tiempos, detección continua para objetos rápidos, y pruebas automáticas de la escena.

20. ¿Quién escribió el código?
Fue elaborado con asistencia de inteligencia artificial a partir de las instrucciones del grupo, apoyado en Ericson (2004), la teoría SAT y la documentación oficial de Godot, y revisado por nosotros. Cada integrante debe poder explicar la parte que le toca.

16. Glosario

Término	Significado
Nodo	Pieza básica de Godot; los nodos forman un árbol.
Escena (`.tscn`)	Conjunto de nodos guardado en un archivo; se puede instanciar muchas veces.
Instancia	Una copia de una escena dentro de otra (las 3 pelotas son instancias de `pelota.tscn`).
Señal	Aviso de un evento (`signal` , `emit` , `connect`).
Grupo	Etiqueta de nodos («jugador», «pelotas»); permite buscarlos o reconocerlos.
`@export`	Variable editable desde el inspector y desde la escena.
`@onready`	Variable que se asigna cuando el nodo está listo (aquí, referencias a hijos).
`class_name`	Da un nombre global a un script para usarlo como tipo.
`_ready` / `_process` / `_physics_process`	Funciones que Godot llama al iniciar / cada cuadro de dibujo / cada paso de física.
`queue_redraw` / `_draw`	Pedir y realizar el dibujo de un nodo con funciones como `draw_rect` .
`queue_free`	Elimina el nodo al final del cuadro.
`Rect2`	Rectángulo con posición y tamaño; se usa para AABB.
`Vector2`	Par (x, y): posición, velocidad o dirección.
Normal	Dirección perpendicular al contacto, usada para separar o rebotar.
Profundidad	Cuánto se solapan dos formas a lo largo de la normal.
PhysicsMaterial	Recurso con fricción y rebote (`friction` , `bounce`).
Headless	Ejecución sin ventana ni gráficos.
DFS	Recorrido en profundidad de un grafo.
Componente conexa	Conjunto de vértices alcanzables entre sí.

17. Autoría y fuentes

El código y la estructura del proyecto se elaboraron con **asistencia de IA (Claude, de Anthropic)** a partir de las instrucciones del grupo. Lo declara la diapositiva 18 y el encabezado de `colision_manual.gd` . **Pendiente del grupo:** anotar qué partes revisó, adaptó o modificó cada integrante (criterio I3 de la rúbrica).

- Teoría de las pruebas: C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann, 2004 (círculos, AABB, SAT).
- SAT: Kansas State University, «Separating axis theorem», CIS 580 Textbook (s. f.).
- Estructura de nodos y señales: documentación oficial de Godot («Physics introduction», «Using Area2D», «Using CharacterBody2D», clase RigidBody2D).
- Referencias completas en formato IEEE: ver la diapositiva 20 de la presentación y la guía completa.